

Unitree G1 Robot Deployment Guide

Mascot Unitree — Hero Pose Kinematic Playback

Version: 1.0 | **Date:** April 2026
Authors: Vinod Kumar, Kim
Hardware: Unitree G1 (29-DOF / 35-motor IDL)

Table of Contents

- 1. Overview
- 2. Hardware Requirements
- 3. Software Requirements
- 4. Safety Rules (Read Before Anything)
- 5. Network Setup
- 6. Step 1 — Robot Boot Sequence
- 7. Step 2 — Arm-Only Test (Gantry, Low Gain)
- 8. Step 3 — Full Body Deployment
- 9. Demo Pipeline (Text Input → Motion)
- 10. Animation Reference
- 11. Troubleshooting
- 12. Emergency Procedures
- 13. Joint Map Reference

1. Overview

This guide walks through deploying pre-recorded hero pose animations (Spider-Man, Iron Man, Hulk, etc.) directly to the Unitree G1 humanoid robot. The pipeline is:

Text Input → Persona Detection → Animation Selection (RAG) → PKL File → Hardware Deployment

Two deployment modes:

Mode	Script	Robot State	Use When
Arm test	g1_arm_replay_airborne.py	Suspended on gantry	First-time validation
Full body	deploy_real.py	Standing, gantry recommended	After arm test passes

All PKL files have been velocity-clamped to ≤ 2.0 rad/s. Do not use unclamped files on hardware.

2. Hardware Requirements

- Unitree G1 robot** (23-DOF model, 35-motor hardware IDL)
- Gantry / suspension rig** — mandatory for first test, strongly recommended for all tests
- Linux laptop/PC** (Ubuntu 20.04 or 22.04) — `unitree_sdk2py` does not work on macOS or Windows
- Ethernet cable** — direct connection from Linux machine to the G1 Ethernet port

- **Two operators** — one at keyboard, one holding the L1+L2 physical e-stop
-

3. Software Requirements

On the **Linux deployment machine**:

```
# Python 3.10 or 3.11
pip install unitree_sdk2py numpy

# Verify the SDK is installed
python3 -c "from unitree_sdk2py.core.channel import ChannelFactoryInitialize;
print('SDK OK')"
```

Copy these files from the repo to the Linux machine:

```
kim_workspace/movements/          ← all 10 velocity-clamped .pkl files
kim_workspace/hardware_deployment/g1_arm_replay_airborne.py
vinod_workspace/deploy_real.py
```

4. Safety Rules — Read Before Anything

These are not suggestions. Skipping any of these can damage the robot or injure people.

1. **Always use a gantry** on the first run of any new animation.
 2. **Two-person rule** — one operates the keyboard, one stays on the L1+L2 hardware e-stop at all times.
 3. **Never run on a floor without first running on gantry.** Even if simulation looked stable.
 4. **hulk_smash and thor_lightning are GANTRY-ONLY.** Both animations fail under gravity in physics simulation. They will tip the robot on the floor.
 5. **Check PKL velocity before running.** All files in `kim_workspace/movements/` have been clamped to ≤ 2.0 rad/s. If you generate new PKLs, run `python3 vinod_workspace/clamp_pkls.py` first.
 6. **Abort immediately** if you hear unusual noise, see unexpected joint oscillation, or if the robot twitches unexpectedly. Press L1+L2.
 7. **Keep humans away** from the robot's arm sweep radius during playback.
-

5. Network Setup

Step 1 — Physical connection

Connect Ethernet cable: Linux machine ↔ G1 Ethernet port (rear of robot).

Step 2 — Set static IP on Linux machine

```
sudo ip addr add 192.168.123.100/24 dev eth0
sudo ip link set eth0 up
```

Or via NetworkManager:

```
nmcli con add type ethernet ifname eth0 ip4 192.168.123.100/24
nmcli con up ethernet-eth0
```

Step 3 — Verify connection

```
ping 192.168.123.1      # should reply from robot
```

If ping fails: check cable, check robot is powered on, check IP range.

Step 4 — Set CycloneDDS environment

```
export CYCLONEDDS_HOME=/opt/cyclonedds # adjust to your install path
export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
```

6. Step 1 — Robot Boot Sequence

Follow this every time before running any deployment script.

Step 1.1 — Power on

Press and hold the power button on the G1 until you hear the startup chime. Wait ~30 seconds for boot.

Step 1.2 — Enter DAMPING mode

On the handheld controller: hold **L2 + A** simultaneously until the robot relaxes all joints to a soft, compliant state. The robot will go limp — this is correct.

DAMPING mode = all motors apply only light damping torque. The robot cannot stand on its own in this mode.

Step 1.3 — Support the robot

If on gantry: attach suspension cables before entering DAMPING mode. Ensure the robot hangs freely with feet off the ground and no cables impeding joint movement.

If on floor (full body test only): have two operators physically support the robot from behind during the 3-second ease-in phase.

Step 1.4 — Confirm DDS is active

On the Linux machine, run the encoder monitor to verify communication is live:

```
cd kim_workspace/hardware_deployment
python3 g1_encoder_monitor.py --interface eth0
```

You should see joint positions streaming at ~500 Hz. If nothing appears after 5 seconds, check the network setup in Step 5.

7. Step 2 — Arm-Only Test (Gantry)

Use this for the **first hardware validation** of any new animation. Only the left arm moves. Legs stay limp (KP=0).

Run command:

```
cd /path/to/Capstone
python3 kim_workspace/hardware_deployment/g1_arm_replay_airborne.py \
  --pk1 kim_workspace/movements/wave_kinematics.pkl \
  --interface eth0
```

What happens:

1. Script prints safety checklist and asks you to type `YES`.
2. Waits for LowState messages from robot (5s timeout).
3. Counts down 3 seconds.
4. Replays **left arm only** (joints 15–21) at KP=20, KD=2.0.
5. Prints frame progress and max arm tracking error every 0.2 seconds.
6. Drops to zero torque when done or on Ctrl+C.

What to watch for:

- Arm should move **slowly and smoothly** — if it snaps or oscillates, hit e-stop
- `max_arm_error` should stay below 0.3 rad — higher means arm is struggling
- Any joint velocity above 8 rad/s triggers automatic abort

To run both arms after left arm passes:

```
python3 kim_workspace/hardware_deployment/g1_arm_replay_airborne.py \
  --pk1 kim_workspace/movements/wave_kinematics.pkl \
  --interface eth0 \
  --both-arms
```

Confirm joint mapping before both-arms:

Verify that PKL joint index 15 (L_shoulder_pitch) physically moves the correct joint. Run the encoder monitor in one terminal while moving joints manually in DAMPING mode and compare indices.

8. Step 3 — Full Body Deployment

Only attempt after the arm test passes cleanly.

Run command:

```
cd /path/to/Capstone
python3 vinod_workspace/deploy_real.py \
  --pk1 kim_workspace/movements/wave_kinematics.pkl \
  --iface eth0 \
  --speed 0.5
```

--speed 0.5 means half-speed. Always start at 0.5 on first run.

What happens:

1. Loads PKL, initializes DDS.
2. **Phase 1 (3 seconds):** Reads current encoder positions, linearly interpolates to first PKL frame. This prevents violent torque snaps from wherever the robot currently is.
3. **Phase 2:** Plays back the full trajectory at 500 Hz with float-indexed interpolation.
4. Drops to zero torque when done.

Safety thresholds:

Parameter	Value
Ease-in duration	3.0 seconds
Leg/Waist KP	200
Arm KP	60
Velocity abort	10 rad/s (any joint)
Control rate	500 Hz

To run at full speed after 0.5x passes:

```
python3 vinod_workspace/deploy_real.py \  
  --pkl kim_workspace/movements/wave_kinematics.pkl \  
  --iface eth0 \  
  --speed 1.0
```

9. Demo Pipeline — Text Input to Motion

To run the interactive demo that takes text commands and plays animations:

On macOS (simulation only — opens pre-rendered video):

```
cd /path/to/Capstone  
source .venv/bin/activate  
python3 main.py --text
```

Then type any of these at the prompt:

```
wave      → wave animation  
flex      → flex pose  
punch     → punch animation  
iron man  → Iron Man repulsor  
spider-man → Spider-Man web shoot  
hulk      → Hulk smash (gantry only)  
captain   → Captain America shield  
thor      → Thor lightning (gantry only)  
wolverine → Wolverine claws
```

On Linux with real robot (hardware deployment):

```
ROBOT_INTERFACE=eth0 python3 main.py --text
```

Setting `ROBOT_INTERFACE=eth0` routes the motion to `deploy_real.py` instead of the video player.

10. Animation Reference

All PKL files are in `kim_workspace/movements/`. All velocity-clamped to ≤ 2.0 rad/s.

Animation	File	Frames	Duration	Floor Safe?
Wave	<code>wave_kinematics.pkl</code>	121	4.0s	Yes
Flex	<code>flex_kinematics.pkl</code>	86	2.9s	Yes
Punch	<code>punch_kinematics.pkl</code>	103	3.4s	Yes
Iron Man Repulsor	<code>iron_man_repulsor_kinematics.pkl</code>	109	3.6s	Yes
Spider-Man Web Shoot	<code>spider_man_web_shoot_kinematics.pkl</code>	91	3.0s	Yes
Spider-Man Landing	<code>spider_man_landing_kinematics.pkl</code>	104	3.5s	Yes
Captain America Shield	<code>captain_america_shield_kinematics.pkl</code>	117	3.9s	Yes
Wolverine Claws	<code>wolverine_claws_kinematics.pkl</code>	87	2.9s	Yes
Hulk Smash	<code>hulk_smash_kinematics.pkl</code>	187	6.2s	GANTRY ONLY
Thor Lightning	<code>thor_lightning_kinematics.pkl</code>	127	4.2s	GANTRY ONLY

GANTRY ONLY animations fail under gravity because `waist_pitch` is passive ($KP=0$) on the 23-DOF model and cannot counterbalance bilateral overhead arm raises.

11. Troubleshooting

"No LowState received" — DDS timeout after 5 seconds

- Check Ethernet cable is plugged in
- Run `ping 192.168.123.1` — if it fails, fix network first
- Check robot is booted (not just plugged in)
- Try `--domain 0` instead of default domain 1

Arm oscillates / shakes during playback

- The joint mapping may be wrong. Run `g1_encoder_monitor.py` and verify index 15 = `L_shoulder_pitch`
- Reduce gains: lower `KP_ARM` from 20 to 10 in `g1_arm_replay_airborne.py`
- Check the PKL has been velocity-clamped (`python3 vinod_workspace/clamp_pkls.py`)

Velocity abort triggered immediately

- Robot may have been moving when script started
- Wait until robot is fully stable in DAMPING mode before running
- Check that pkl was clamped: `max_delta * 30` should be ≤ 2.0 rad/s

Robot does not move at all

- Confirm DAMPING mode is active (L2+B on controller)
- Confirm `mode_pr` and `mode_machine` match your robot firmware version
- Check CRC: the LowCmd_ must have valid CRC or robot ignores it

"PKL missing 'joint_angles' key"

- You are using a PKL in the Mascot Unitree format (`dof_pos` , `root_pos` , etc.)
- Use only PKLs from `kim_workspace/movements/` — these have the `joint_angles` key in 35-DOF hardware format

PKL has wrong DOF (not 35)

- Run: `python3 -c "import pickle, numpy as np; d=pickle.load(open('your.pkl','rb')); print(np.array(d['joint_angles']).shape)"`
- Must be `(N, 35)` . If it is `(N, 29)` , the file is from MuJoCo simulation and needs retargeting

12. Emergency Procedures

During playback — something goes wrong:

1. **Operator 2:** Press and hold **L1 + L2** on the controller immediately
2. **Operator 1:** Press **Ctrl+C** in terminal — script sends zero-torque command automatically
3. Physically support the robot until it is fully limp
4. Do not power off unless necessary — preserves DDS state for diagnosis

After e-stop:

1. Note which joint/frame triggered the problem
2. Check `g1_encoder_monitor.py` output for any stuck joints
3. Do not retry until root cause is identified

If robot falls:

1. Do not attempt to catch — step back
2. Power off robot after it stops moving
3. Visually inspect all joints before next power-on

13. Joint Map Reference

G1 Hardware IDL — 35-motor layout:

Index	Joint Name	Group
0	L_hip_pitch	Left Leg
1	L_hip_roll	Left Leg
2	L_hip_yaw	Left Leg
3	L_knee	Left Leg

4	L_ankle_pitch	Left Leg
5	L_ankle_roll	Left Leg
6	R_hip_pitch	Right Leg
7	R_hip_roll	Right Leg
8	R_hip_yaw	Right Leg
9	R_knee	Right Leg
10	R_ankle_pitch	Right Leg
11	R_ankle_roll	Right Leg
12	waist_yaw	Waist
13	waist_roll	Waist (PASSIVE in 23-DOF)
14	waist_pitch	Waist (PASSIVE in 23-DOF)
15	L_shoulder_pitch	Left Arm
16	L_shoulder_roll	Left Arm
17	L_shoulder_yaw	Left Arm
18	L_elbow	Left Arm
19	L_wrist_roll	Left Arm
20	L_wrist_pitch	Left Arm
21	L_wrist_yaw	Left Arm
22	R_shoulder_pitch	Right Arm
23	R_shoulder_roll	Right Arm
24	R_shoulder_yaw	Right Arm
25	R_elbow	Right Arm
26	R_wrist_roll	Right Arm
27	R_wrist_pitch	Right Arm
28	R_wrist_yaw	Right Arm
29–34	Extended joints	Zero torque (unconfirmed mapping)

Note on waist_pitch (index 14): In the 23-DOF MuJoCo model, `waist_pitch` is passive — commands are ignored. This is why `hulk_smash` and `thor_lightning` cannot maintain balance on the floor. The hardware IDL does have this motor, but the robot's firmware treats it as passive.

End of Guide

For issues or questions, file a [GitHub issue](#) at [K-vinod2k/Capstone-Project](#)